OXFORD

The Computer Journal, 2024, **67**, 3160–3168
https://doi.org/10.1093/comjnl/bxae079
Advance access publication date 31 August 2024
Original Article

Graph neural network based phishing account detection in Ethereum

Siftee Ratra[1], Mohona Ghosh[1], Niyati Baliyan[2,*], Jinka Rashmitha Mohan[1], Sanjana Singh[1]

[1]Department of Information Technology, Indira Gandhi Delhi Technical University for Women, New Church Rd, Kashmere Gate, Delhi, 110006 Delhi, India
[2]Department of Computer Engineering, National Institute of Technology Kurukshetra, Thanesar, Haryana, 136119 Kurukshetra, India

*Corresponding author. E-mail: niyatibaliyan@nitkkr.ac.in

Abstract

In recent years, the widespread adoption of Ethereum-based transactions, such as cryptocurrencies and blockchain technologies, have revolutionized the way financial transactions are conducted. These decentralized and transparent systems offer numerous advantages, including enhanced security, immutability, and reduced transaction costs. However, alongside their benefits, Ethereum-based transactions have also attracted the attention of malicious actors seeking to exploit unsuspecting users through phishing scams. Phishing scams have thus become frequent in this scenario. Therefore, it is required to implement an effective and reliable phishing scam detection method. In this paper, we present the implementation of a highly efficient detection method by carrying out a graph-like data network formation, over which we then apply models that are based on graph neural networks like Magnet Link Prediction and Graph AutoEncoder Pathfinder Discovery Network Algorithm (GAE_PDNA). This helps in extracting useful information from the nodes of the graph. After relevant embeddings have been obtained, the classification of the phishing account is performed using AdaBoost classifier that helps in complex decision-making and detects the accounts related to the phishing scams. Our best model attains a precision of 0.99 and an F1 score of 0.99.

Highlights

- The extracted transaction network of Ethereum contains very few phishing accounts (which need to be detected to prevent fraudulent activities) compared to the non-phishing ones.
- SMOTE followed by random sampling has been utilized to handle the imbalance.
- It is shown that Graph neural networks can be used efficiently and accurately to learn useful features of the network for prediction.
- Experiments reveal that ensemble learning-based classifiers are superior at classifying accounts in the network.

Keywords: graphs; neural Networks; phishing; Ethereum; blockchain; classification

1. INTRODUCTION

Blockchain (https://www.synopsys.com/glossary/what-is-blockchain.html) is a decentralized database that is unchangeable and enables easier tracking of the transactions records. Ethereum (https://ethereum.org/en/what-is-ethereum/) is the largest open source decentralized blockchain software platform that is used to build a safe technology that can be used for recording transactions. Ethereum is secure, scalable in nature as well as programmable. It helps in transitioning the way of doing work in many sectors.The widespread popularity enjoyed by Ethereum has also made it a lucrative target for different kinds of crypto crimes—*phishing scams* being one of the notable crimes amongst them [1].

Phishing (https://www.proofpoint.com/us/threat-reference/phishing) attacks are a kind of engineering attack where the attacker masquerades as a trustworthy institution and tries to dupe the chosen victim to reveal sensitive information or be exposed to malignant software. Phishing scams have become an increasingly prevalent and concerning issue in today's digital landscape. The consequences of falling victim to a phishing scam can be severe. Individuals may experience financial loss, identity theft, unauthorized access to their accounts, and even reputation damage. Thus, any observed occurrence of phishing-related scams in the Ethereum community must be dealt with promptly and efficiently. Detecting phishing scams requires a multi-faceted approach that combines technological advancements, user education, and proactive security measures. We suggest a unique approach to detect phishing scams. The first stage is data crawling and forming a graph-like transaction network through network embedding where nodes depict the accounts and edges depict the transactions between the accounts. This graph helps in capturing complex relationships between the networks and in analyzing complex networks and detecting dynamic patterns of the data [2].

After the data aggregation is completed, we apply models like magnet link prediction and GAE_PDNA that are based on Graph Neural Networks (GNNs). GNNs have emerged as a transformative approach in the field of machine learning and data analysis, providing a very strong framework for modeling and understanding complex graph-structured data. With their ability to capture intricate relationships and dependencies within graphs, GNNs have gained significant attention and have been successfully applied across various domains [3].

One of the key strengths of GNNs lies in their capability to capture complex relationships. Unlike traditional approaches that treat graph data as independent instances, GNNs explicitly model the inter-dependencies between nodes, enabling them to capture

Received: October 5, 2023. **Revised:** June 29, 2024. **Accepted:** August 22, 2024
© The British Computer Society 2024. All rights reserved. For permissions, please e-mail: journals.permissions@oup.com

Downloaded from https://academic.oup.com/comjnl/article/67/12/3160/7746724 by National Institute of Technology Kurukshetra user on 05 February 2026

rich semantic information and dependencies among connected entities. This makes GNNs particularly well suited for applications where the relationships between entities play a crucial role, such as social network analysis, recommender systems, and knowledge graphs [4].

Additionally, GNNs have the potential to scale and generalize, enabling them to manage massive graphs with millions or even billions of nodes and edges. Their ability to generalize knowledge learned from one graph to new, unseen graphs makes GNNs valuable in scenarios with limited labelled data, as they can leverage knowledge from related graphs to make predictions.

Overall, the current work aims to use GNN models that help us identify the phishing scams in the best possible way. We present a novel GNN architecture combined with AdaBoost classifier to detect phishing scams. Our aim is to propose a model that is very reliable and efficient for detecting phishing scams in real-world scenarios and is capable of scaling according to the requirements.

2. RELATED WORK

Amidst the rapid growth of Ethereum, there is a rise in the number of online scams and crimes that occur through its medium. There are numerous articles available online and in print that indicate how susceptible blockchain really is to fraudulent occurrences. (https://blog.chainalysis.com/reports/ethereum-merge-scams/), (https://www.bbc.com/news/uk-england-manchester-60465675) In fact, these articles point to the fact that, in an approximate measure, 10% of the invested Ether has actually been stolen by malicious members of the blockchain community. The frauds can be further divided into several sub types [5]:

- Scam wallets
- Ponzi schemes
- Fraudulent exchanges

Significant research has been done to find patterns to detect the aforementioned scams that occur in Ethereum.

In [6], Bartolett *et al.* conducted a comprehensive survey of Ponzi schemes on Ethereum, studying their behaviour and their impact from multiple aspects. In [7], mining of the blockchain data and usage of applied machine learning algorithms was done by the authors to identify Ponzi schemes. This work may expand its ground truth,thereby improving classification performance. Further, currently, a website for their solution does not exist and the only scams handled are smart Ponzi schemes.

Our paper targets phishing attacks wherein a malicious website pretends to be a sincere organization and, in the process, acquires crucial and sensitive information of the users like their passwords or identification numbers. Many prior papers have been published to identify and act against phishing scams.

Abdelhamid *et al.* in [8] targeted website phishing and for the same, a proposal was made to use multi-label classifier to generate a set of rules based on the frequency analysis of the features of the whole dataset. Since the feature assessment of huge collection of website features has been carried out, it may have computational overhead. Authors in [9] used Support Vector Machine (SVM) classifier to detect phishing websites based only on the URL links of the websites while ignoring HTML features, search engines, Image recognition, and DNS look up. Probabilistic prediction on the phishing websites has not been explored, in addition, the number of records may be scaled up from 2000 to represent real-world scale. Researchers in [10] proposed an approach to combat website phishing scams wherein visual inspection was done of the dubious web pages with the secure ones. However, the employed graph representation model based on temporal random walk is a non-inductive method, and hence unable to obtain the representations for the newly added nodes. Furthermore, authors plan to develop more specific graph representation methods to explore different various predictive tasks on Ethereum. Yuan *et al.* in [11] proposed the idea of representing transaction records as directed graphs and using the node2vec network embedding approach for the final classification. The manually made features using basic transaction information may be effective yet inefficient in manifesting the properties of subgraphs. Complex methodologies such as graph convolutional network (GCN) and autoencoders were first utilized by Chen *et al.* in [12]. Graph network specific approaches such as graph-based cascade feature extraction were used by authors in [13]. They also utilized ensemble learning for the detection of fraudulent accounts. Currently, this work does not cover other cybercrimes in its scope and a website or API like service may also be a future work. Wen *et al.* in [14] proposed a unique solution with two frameworks: one for feature learning and a hidden one for the addition of records of the transaction.

Table 1. Node attributes

Node attributes
Balance value in Ether
Total transactions made to the node
Total transactions made from the node

Ogundokun *et al.*, in [15], have implemented Blockchain for transactional addresses which are then preprocessed and shown numerically through word embedding transformations. This is followed by classification using different long short-term memory (LSTM) based neural network models. Ensemble voting has been finally used to classify the transaction as either malicious or benign. Limitations include under exploration of feature extraction and scaling, and in a decentralized medium. Zhang *et al.* in [16], put forward the idea of a multi-channel graph classification model (MCGC) wherein there are multitude channels where feature extraction takes place for the final GNN model. In [17], the node and multi-edge features are taken into account by the authors Kim *et al.* to convert the input transaction network into a directed and heterogeneous graph which they named as 'ATGraph'. Afterwards, various graph learning algorithms are applied on this new graph. However, the authors did not consider externally owned account types in their work. Further, one may consider various node/edge types to reflect the heterogeneity of the Ethereum network better. In [18], authors Zhou *et al.* first retrieve the list of transactions for a tagged account and later construct a star-shaped subgraph of the entire transaction network for that account.

In our paper, we use auto-encoders for embedding, which facilitates the extraction of comprehensive feature representation. In contrast, we observe that most of the previous research works that have utilized graph embedding, relied on walk-based embedding algorithmic strategies.

Our inventive amalgamation of GNNs, Auto-encoders, and ensemble learning epitomizes a revolutionary strategy for Phishing Account Detection within the Ethereum realm. This approach embodies novel methodologies, innovative paradigms, and novel solutions, which push the boundaries of conventional detection systems.

Downloaded from https://academic.oup.com/comjnl/article/67/12/3160/7746724 by National Institute of Technology Kurukshetra user on 05 February 2026

Table 2. Edge attributes

Timestamp
Amount (in Ether)
Block height

3. DATASET FORMATION AND DESCRIPTION

- **Formation**
  We have used Etherscan [19] and Cryptoscamdb (https://cryptoscamdb.org/) to extract the accounts which are categorised as 'phishing' in their respective web pages. We used Selenium (https://www.selenium.dev/) to scrape the data from both the websites and collated them accordingly. We were able to scrape 700 transaction accounts from Etherscan and 648 from Cryptoscamdb. A total of 898 phishing accounts were collected upon merging. Web crawling was then used to extract transactions of the accounts, with these phishing accounts as starting nodes. Further queries were continued onto the neighbour nodes. We performed this till a recursive depth order of 2.
  Thus, the network so obtained consisted of total 1 088 627 nodes (i.e., addresses) and 1 441 830 edges (i.e., transaction relationship between two addresses). Out of these, 898 nodes were labelled as phishing nodes.
  As there can be a transaction information where the recipient and sender are interconnected with many other such recipients and senders, hence, there is a many-to-many relationship that can be collated into graph format and so we have transformed the obtained data into graph format using NetworkX graph [20].
- **Description**
  In our graph format, the nodes represent transaction accounts and the edges between two nodes indicate a transaction being performed between the two nodes with indicative features like ' TimeStamp', ' BlockHeight', and ' Value' being edge attributes. We construct a directed graph for the same. We also have allocated each node a set of three attributes, i.e., the account balance of the node and the in-degree and out-degree of the node. Table 1 and 2 summarize the final extracted node and edge attributes of the graph.
- **Handling data imbalance** There is a great variation between the phishing accounts data and the overall transaction dataset. To combat this, we have used SMOTE [21] to perform oversampling and subsequent undersampling and generate a balanced dataset of phishing and non phishing accounts to handle the size mismatch.

4. DATASET EXPLORATION

The Networkx graph formed consisted of total 1,088,627 nodes and 1,441,830 edges. Figure 1 shows the degree, in-degree, and out-degree rank of matrix.

Due to the sheer size of this network, more exploration and analysis was difficult using only Networkx and alternative graph analysis tools were also utilized to understand the data. One such tool initially used was Gephi [22]. The graph was queried for its degree distribution. The average degree of nodes was found to be 1.324. From the graphs procured, it was inferred that the maximum degree in the graph reaches 10 219. A big chunk of nodes (about 15%) possess a degree of 0. The spread of other present values is consistently low. The weighted degree distribution also yielded the same distributions and average value as the normal degree distribution. The rest of the stated computations have been done using igraph [23], owing to its superior performance in terms of speed and RAM usage. The degree assortativity of the graph was found be to −0.100. The negative value indicates a tendency of association of dissimilar nodes in the graph. However, the value being close to 0 signifies a largely neutral network. The average number of triangles in the graph was evaluated to be 0.103 and the maximum number of triangles was 7194. In order to see the extent of the graph, processing was also done to find the diameter and average path length which came out to be 19 and 6.041, respectively.

4.1. Dataset comparison with related work

In this section, to understand the dataset further, we do a comparative analysis of its characteristics with what we considered to be the six most relevant datasets from the related work discussed in section 2. The comparison is done in Table 3. Please note |R| stands for Paper Reference, |S| stands for Source of dataset, |E| stands for the number of edges in the network, |V| stands for the number of vertices, |P| is the number of labelled phishing addresses, and |N| is the labelled non-phishing addresses utilized in the final binary classification to detect phishing addresses.

A common pattern that can be gleaned from the datasets mentioned in Table 3 is the sheer size of the transaction network (Row 1 is an exception). Only a small subset of the total nodes in the network are categorised as phishing nodes. Row 1 and Row 4 are the only works that deal with the inherent data imbalance directly. Our first proposal, Magnet Link Prediction (6.2.1), also deals with the data imbalance directly.

In all the other cases mentioned, an equal number of non-phishing nodes is pre-decided from the network for the purpose of the final binary classification. This is their way of dealing with the imbalanced data. For this reason, we give an answer of 'No' to the 'Data imbalance?' question proposed in Table 3 for these cases.

Our second proposal, GAE_PDNA (6.2.2) has employed a somewhat similar tactic in this regard. However, rather than choosing a random equal number of non-phishing nodes as outliers, we utilized SMOTE to equalize the dataset instead. The node and edge features utilized for the purpose of learning from the formed network greatly varies across the papers, with some studies utilizing only edge features and some studies utilising both, and,thus, omitted from Table 3.

5. MODELS

5.1. Embedding Models

5.1.1. *DeepWalk*

Deepwalk [29] is a random walk technique where latent vertices representations can be learnt from a network. In order to properly learn local transaction networks, it applies random search techniques. This is achieved by using these random paths as sequences to train a skip-gram language model.

As we optimize the certainty of phrases that occur within a frame, we make use of the Skip-Gram Model [30] to get the probability of the next word in the phrase.

- Skip-Gram Model
- In the skip-gram model, given a word it predicts the neighbouring words. This model is accommodated to find the features of a node, given a node.

Downloaded from https://academic.oup.com/comjnl/article/67/12/3160/7746724 by National Institute of Technology Kurukshetra user on 05 February 2026

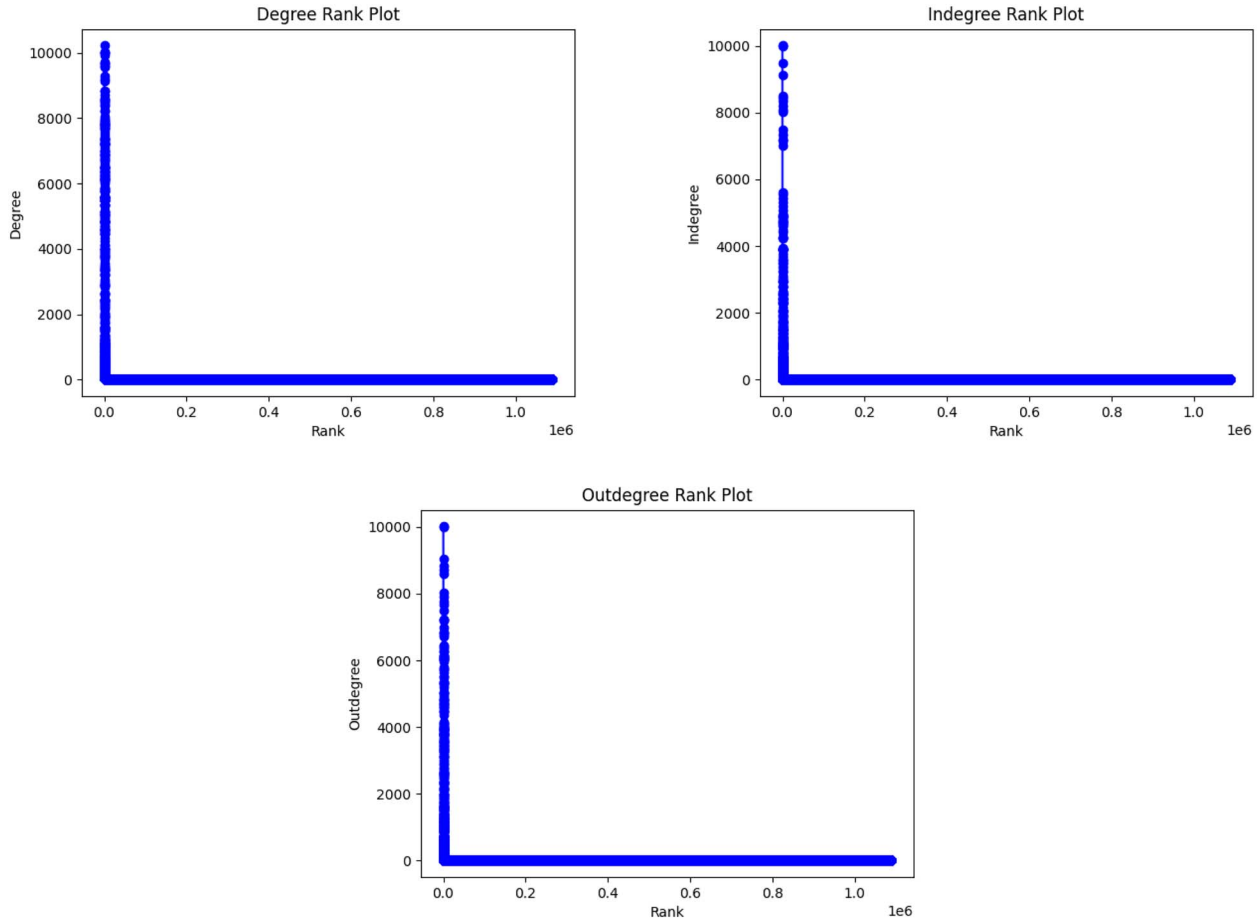

Figure 1. Degree ranks.

Table 3. Dataset comparative analysis

|R|	|S|	Extraction technique	|E|	|V|	|P|	|N|+	Data imbalance?
[15]	[24]	List of both dark-listed and whitelisted addressees is in JSON format, hence is easily extractable.	-	2407	2373	34	Yes
[17]	[25]	Normal and phishing addresses extracted from XBlock [25], using them as central nodes, web crawling fetches the list of transactions, and forms transaction network.	22 0714	111 956	1659	1700	No
[18]	[19]	Normal and phishing addresses' extraction and using them as central nodes, web crawling fetches the list of transactions. A star-shaped sub-graph of the network is made.	343 342	641 568	1659	1700	No
[7]	[26] and [19]	Ethereum client Parity [26] gets Ethereum blocks in a fixed block height range. Web crawling using Etherscan [19] labels phishing addresses. Filtering rules applied for data balancing.	7 795 044	534 820	323	534 497	Yes
[1]	[27] and [19]	Phishing addresses extracted from [19] and [27]. An equal number of unlabelled addresses randomly selected as non-phishing. Using these addresses as central nodes, web crawling fetches the list of transactions for each node with traversal order 1.	200 000	60 000	1259	1259	No
[28]	[19]	Phishing addresses extracted from Etherscan [19]. Equal number of unlabelled addresses randomly selected as non-phishing. Web crawling uses traversal order 2.	13 551 214	2 973 382	1071	1071	No

Downloaded from https://academic.oup.com/comjnl/article/67/12/3160/7746724 by National Institute of Technology Kurukshetra user on 05 February 2026

5.1.2. *Node2Vec*

Node2vec [31] is a machine learning algorithm to procure node embeddings in a graph, which can, then, be used for various tasks such as node classification, link prediction, and visualization. These are representations of nodes in a graph converted to a lower dimension that capture their structural and semantic similarities.

The algorithm learns node embeddings by optimizing the likelihood of observing the graph structure given the learned embeddings. The algorithm achieves this by learning a function that maps nodes to embeddings based on the network structure.

Node2vec is an adaption of the skip-gram model from NLP but adapts it for graph data. It utilizes a biased random walk to generate node sequences that seeks to preserve any global as well as local structural information in the graph.

5.1.3. *Trans2Vec*

Trans2vec [1] is a biased random walk network embedding model, whose main function is to compress transaction information into representation vectors for the node. Precisely, a traversal through r random walks with l walk distances is done from each starting node. Biased sampling technique has been created that would help in easily switching between the biases based on transaction size and timestamp at each stage of the walk. It was observed that the inclusion of amount and time-based parameters while classification improved the accuracy, indicating that a correlation exists between phishing tendency and the time and amount of transaction.

5.1.4. *Graph Autoencoder Model—GAE_PDNA*

We have seen that in images, autoencoders are used to learn features of a particular image and then the learned features are later used to regenerate similar images or identify similar images to perform classification. Here we use the same method by utilizing graph neural network layers wherein we properly analyse the nodes and identify the distinctive attributes of phishing accounts that make them unique amongst a sea of nodes. A Graph Autoencoder (GAE) is a type of neural network structure created to grasp low-dimensional representations of graphs as can be seen in Figure 2. Its aim is to compress a graph into a lower-dimensional vector (or latent space), then reconstruct the original graph as accurately as possible from this vector. GAEs utilize unsupervised learning, meaning they extract valuable features from input graphs without direct guidance or supervision. GAEs comprise two primary elements: an encoder, which condenses the graph into a low-dimensional vector, and a decoder, which reconstructs the original graph from the vector. The encoder starts with the input graph and employs multiple graph convolutional layers. These layers capture both local and global graph structures, extracting key features. The final output of the encoder is a low-dimensional embedding, representing the graph's encoded form.

The decoder utilizes the encoded representation and reverses the process by applying a sequence of graph convolutional layers. These layers aim to reconstruct the original graph. The final output of the decoder's last convolutional layer is the reconstructed graph, striving to closely match the input graph.

The goal of the graph autoencoder is to minimize the difference between the input graph and the reconstructed graph using a loss function. By minimizing this difference, the encoder learns to capture the essential features of the input graph in the encoded representation, while the decoder learns to reconstruct the graph from the encoded representation. GAEs can be used for a variety of tasks, such as node classification, link prediction, and anomaly detection in graphs. Since our goal was anomaly detection, so to identify the phishing accounts, we used this method. We built our new model- GAE_PDNA, which is a graph autoencoder using Pathfinder Discovery Networks (PDN) [32] layers internally.

PDN have each node learning the aggregate weights put together from all the connected edges of that particular node. This enables PDN to learn better by assessing all the immediate neighbours' relations. The problem of diminishing weight, where with increasing nodes, the weight starts to tend to zero, has also been counteracted against very well. PDNs can be used with GNNs for both supervised or unsupervised learning. In our model, the single unit of operation consists of a PDNConv layer, followed by a PreLU operation [33] and a conclusion by applying graph normalization using GraphNorm [34]. We repeat this unit 4 times. The output is formed by applying a last PDNConv operation on the final result.

PDNConv makes use of not only node attributes but edge attributes as well. There are three node attributes associated with the accounts as already explained in Table 1. These are the input channels. The dimension of the output channel is set to 15. Apart from this, for the three edge attributes (Table 2), hidden channel dimension has been set constant to 6. For the architecture, we drew inspiration from the anomaly detection model proposed in [35], adapting it to our specific use case by adjusting parameters and dimensions. Figure 3 gives a visual representation of the model described.

5.2. Classification

5.2.1. *Link prediction*

A major challenge we faced while trying to handle the graph was that it was not bidirectional. Hence, conventional methods were to be modified accordingly to analyse the directed graph. We used MagNet [36], a GNN, specially used to apply on the directional graphs, that utilizes a complex Hermitian matrix called the magnetic Laplacian. MAGNet (Multi-task Graph Attention Network) is a GNN model that can be used for link prediction in graphs. The model employs an attention mechanism to understand the importance of different nodes and edges within the graph. Additionally, it is structured to handle multi-task learning, allowing it to simultaneously learn multiple related tasks.

To utilize MAGNet for link prediction, the initial stage entails representing the graph effectively. This could entail converting the graph into a feature matrix or adjacency matrix, depending on the implementation. Subsequently, the MAGNet model is trained on the graph data utilizing a suitable loss function, such as cross-entropy or binary cross-entropy loss.

Throughout the training phase, the model grasps how to forecast the likelihood of a link existing between any two nodes within the graph. Following training, the model can assess links in new or unseen graphs by inputting pertinent graph data and employing the trained model to anticipate the probability of links between various pairs of nodes.

The performance of MAGNet for link prediction can depend on various factors such as the specific graph data and the training parameters used. However, MAGNet has been shown to achieve high results on various benchmark datasets for link prediction tasks.

5.2.2. *AdaBoost*

AdaBoost has been shown to be effective in a variety of applications, including face detection, text classification, and spam filtering.

Downloaded from https://academic.oup.com/comjnl/article/67/12/3160/7746724 by National Institute of Technology Kurukshetra user on 05 February 2026

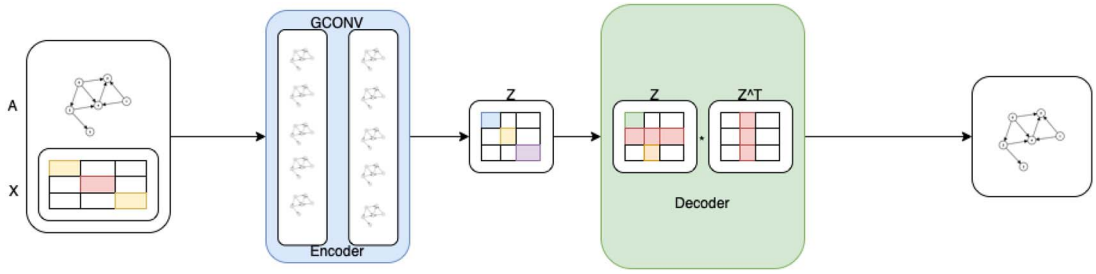

Figure 2. Processing by a graph autoencoder.

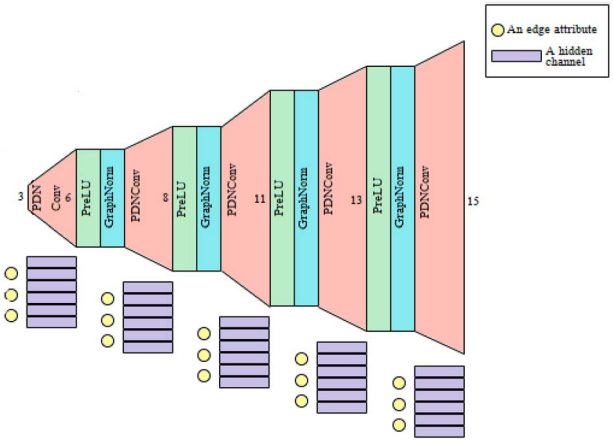

Figure 3. The GAE_PDNA-GNN.

AdaBoost, or Adaptive Boosting, is employed in classification and regression tasks. Its core concept involves amalgamating weak learners—classifiers that marginally outperform random guessing—into a robust learner with significantly improved accuracy. This amalgamation is weighted, with misclassified examples receiving higher weights and correctly classified ones lower weights. Ultimately, the weighted combination of weak learners contributes to the final prediction.

The algorithm operates through iterative inclusion of weak learners into the ensemble, adjusting the weights of training examples according to misclassifications. Each iteration involves selecting a new weak learner that minimizes the weighted error rate on training examples. Furthermore, the accuracy of each weak learner determines its weight within the ensemble, with updates made accordingly. We applied this classifier to the encoded embeddings obtained from GAE_PDNA and received phenomenal results which will be explained in the later section.

5.2.3. *Support vector classifier*

We consider phishing accounts' information as a form of outlier or an anomaly since the dataset is highly imbalanced, with the number of phishing accounts being very meagre and positive result for phishing scam being scanty. Thus, it is safe to assume that phishing accounts nodes can be considered as outliers. SVM turns this into a single class classification and rest as outliers. This can be attributed to the hyperplane that determines the distinction between the valid or authenticated accounts.

5.2.4. *Naives Bayes classifier*

This algorithm relies on Bayes' Theorem for classification. It calculates the probability of occurrence based on the given evidence. The Naive Bayes classifier assumes that there is an independence between features of a data point. This assumption simplifies the calculation of the probabilities involved in Bayes' theorem and makes the algorithm computationally efficient.

The fundamental premise has been that each feature has independent and equal contribution to the result.

5.2.5. *Random forest classifier*

Random Forest Classifier is an ensemble learning algorithm used for classification tasks in machine learning. It is based on decision trees and combines multiple decision trees to make a more accurate prediction. During training, the random forest classifier builds multiple decision trees. Each tree is formed by selecting a random subset of the training data and features. This randomization aids in reducing overfitting and improving the accuracy of the model.

5.2.6. *Logistic regression*

It is a type of categorization/classification method. Construction of the dependent variable is done using a logistic function. It consists of two variables amongst which, one is dependent on the other and it ranges from 0 to 1, making it perfect for binary classification.

6. RESULTS

The experiment was conducted with the final graph constructed and the performance of different models was tested and verified. Section 6.1 lists the relevant metrics utilized for the same. Section 6.2 documents the final results of proposed models as well as does the comparative analysis.

6.1. Evaluation metrics

The task of predicting phishing accounts was modeled as a binary classification problem and thus a node in our work can belong to one of the two possible labels: 1 if the node is a phishing account and 0 if it is a non-phishing account.

As already pointed out in section 4, the graph contains 1 088 627 nodes. Only 898 nodes out of these are phishing accounts i.e., have the label 1. Thus, there is a severe data imbalance problem with 0 being the majority class that threatens to overshadow the minority class 1.

Therefore, conventional measures of accuracy and loss cannot be used for assessment of the performance of the models. Instead, we make use of four better suited metrics: precision, recall, F1 score, and AUC or Area under Curve, higher the AUC, the more

Downloaded from https://academic.oup.com/comjnl/article/67/12/3160/7746724 by National Institute of Technology Kurukshetra user on 05 February 2026

Table 4. Magnet link prediction

Class	Trained Data			Test Data		
	Prec.	Rec.	F1	Prec.	Rec.	F1
0	0.88	0.91	0.88	0.87	0.88	0.87
1	0.90	0.88	0.88	0.88	0.87	0.87
Micro-average	0.88	0.89	0.88	0.87	0.87	0.87

robust the classifier.

$$Precision = \frac{TP}{TP + FP} \tag{1}$$

$$Recall = \frac{TP}{TP + FN} \tag{2}$$

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall} = \frac{2 \times TP}{2 \times TP + FP + FN} \tag{3}$$

6.2. Performance of different models

The node and edge attributes associated with the dataset are already described in Tables 1 and 2. Owing to the wide variance prevalent in the balance values (given in ether) of the nodes, log transformation was applied to reduce the range of this attribute value. All the attributes were then individually normalized. This was the mandatory pre-processing done before any detection or embedding was performed. Sections 6.2.1 and 6.2.2 provide the implementation details and results obtained by using our two proposed models, Magnet Link Prediction and GAE _PDNA.

6.2.1. *Magnet Link Prediction*

PyTorch Geometric [37] comes with its in-built implementation of the MagNet link classification model. We have used the same for our implementation. We got the best results by setting the hidden channels to 8 and setting the q parameter to be trainable (initialized to 0.15). In order to reduce occurrences of overfitting, the dropout rate was increased to 0.8 from the default value of 0.5. The rest of the parameters used their default values. The training was done for 100 iterations. The results obtained are shown in Table 4.

6.2.2. *GAE_PDNA*

Before running the GAE_PDNA autoencoder, a random link split was performed in a 70:10:20 ratio. Reconstruction loss was back-propagated in each training iteration and training was performed 200 times. The final training, validation, and testing loss generated by the encoder was 0.7599, 0.7589, and 0.7590, respectively. The node embeddings of the equalized dataset were presented in two-dimensions using TSNE algorithm [38] and are shown in Figure 4,

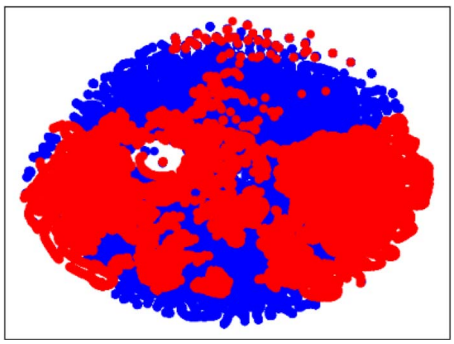

Figure 4. Node embedding obtained on the equalized dataset.

wherein red dots are the phishing accounts, whereas blue are normal ones.

After the autoencoder had been trained, the network was sent to be encoded and consequent node embeddings were obtained. The shape of the node embeddings was 1 088 627(number of nodes in graph) × 15(number of output features). To counter the data imbalance problem, over-sampling was done on the minority class such that it becomes 10% the size of the majority class. Subsequently, random undersampling of the majority class was done to maintain a 1:1 ratio. Therefore, now we had a total of 108773 phishing and non-phishing nodes each respectively. The balanced dataset was then split in a 80:20 ratio and sent to the final detection layer which was an Adaboost classifier. The base estimator used for the same was decision tree classifier and 100 was chosen as the limit at which boosting would be terminated. The final results obtained are shown in Table 5. As we can see, the results are far superior. Finally, we can see the final AUC score of our models in Table 6.

6.2.3. *Comparative study*

Table 7 does a comparative study of our two models with other relevant approaches, differing with respect to the embedding algorithms used. We use the best performance of each model for our comparison.

Table 8, on the other hand, compares the performance of using classifiers other than Adaboost on encoded values obtained by

Table 5. GAE_PDNA

Class	Trained Data			Test Data		
	Prec.	Rec.	F1	Prec.	Rec.	F1
0	1	0.98	0.99	0.99	0.98	0.99
1	0.98	1	0.99	0.98	0.99	0.99
Micro-average	0.99	0.99	0.99	0.99	0.99	0.99

Downloaded from https://academic.oup.com/comjnl/article/67/12/3160/7746724 by National Institute of Technology Kurukshetra user on 05 February 2026

Table 6. AUC Score

Model	AUC Score
MagNet	87.15
GAE_PDNA	98.95

Table 7. Performance comparison: embedding algorithms

Model	Precision	Recall	F1-score
Deepwalk	0.65	0.65	0.65
Node2Vec	0.85	0.81	0.83
Trans2Vec	0.92	0.88	0.90
MagNet	**0.87**	**0.87**	**0.87**
GAE_PDNA	**0.99**	**0.99**	**0.99**

Table 8. Performance comparison: classifier algorithms

Classifier	Precision	Recall	F1-score
Naive Bayes	0.71	0.81	0.69
One-class SVM	0.88	0.89	0.88
Logistic Regression	0.89	0.90	0.89
Random Forest	0.98	0.98	0.99
Adaboost	**0.99**	**0.99**	**0.99**

GAE_PDNA. Both Adaboost and random forest classifier reached the highest scores, proving the effectiveness of ensemble learning for this use-case.

7. CONCLUSION

In this work, we propose a methodical approach to identify and detect phishing scam accounts in the Etherscan network, right from the fetching and procurement of the dataset (using web scraping and web crawling) to its processing and synthetic data generation to the final detection process using graph neural networks. Using a combination of graph autoencoders and ensemble learning-based classifiers has helped us achieve an improvement of about 9% in F1-score during classification, when compared with previous best work that utilizes the Trans2Vec embedding approach under similar experimental conditions. This is a significant empirical advancement in the domain of phishing account detection by graph neural network-based approach.

Data availability

The datasets generated during the current study are available at https://github.com/Siftee150/MajorProject.

References

1. Wu J, Yuan Q, Lin D. *et al.* Who are the phishers? Phishing scam detection on Ethereum via network embedding. *IEEE Trans Syst Man Cybern Syst*. 2022; **52**:1156–1166. https://doi.org/10.1109/TSMC.2020.3016821.
2. Wu H, Song C, Ge Y. *et al.* Link prediction on complex networks: an experimental survey. *Data Sci Eng*. 2022; **7**:253–278. https://doi.org/10.1007/s41019-022-00188-2.
3. Zhou J, Cui G, Hu S. *et al.* Graph neural networks: a review of methods and applications. *AI Opesn*. 2020; **1**:57–81. https://doi.org/10.1016/j.aiopen.2021.01.001.
4. Dong X, Zhang Y, Pang K. *et al.* Heterogeneous graph neural networks with denoising for graph embeddings. *Knowl-Based Syst*. 2022; **238**:107899. https://doi.org/10.1016/j.knosys.2021.107899.
5. Vasek M, Moore T. There's no free lunch, even using bitcoin: tracking the popularity and profits of virtual currency scams. In: Böhme R, Okamoto T (eds.), *Financial Cryptography and Data Security*. Berlin: Springer-Heidelberg, 2015. https://doi.org/10.1007/978-3-662-47854-7_4.
6. Bartoletti M, Carta S, Cimoli T. *et al.* Dissecting Ponzi schemes on Ethereum: identification, analysis, and impact. *Future Gener Comput Syst*. 2020; **102**:259–277. https://doi.org/10.1016/j.future.2019.08.014.
7. Weili C, Zibin Z, Jiahui C. *et al.* Detecting Ponzi schemes on Ethereum: Towards healthier Blockchain technology. In: *WWW'18: Proceedings of the 2018 World Wide Web Conference, Lyon, France, 23–27 April*, pp. 1409–1418. Republic and Canton of Geneva, CHE: International World Wide Web Conferences Steering Committee, 2018.
8. Abdelhamid N, Ayesh A, Thabtah F. Phishing detection based associative classification data mining. *Exp Syst Appl*. 2014; **41**: 5948–5959. https://doi.org/10.1016/j.eswa.2014.03.019.
9. Zouina M, Benaceur O. A novel lightweight URL phishing detection system using SVM and similarity index. *HCIS*. 2017; **7**:1–13. https://doi.org/10.1186/s13673-017-0098-1.
10. Lin D, Wu J, Yuan Q. *et al.* Modeling and understanding Ethereum transaction records via a complex network approach. *IEEE Trans Circuits Syst II: Express Briefs*. 2020; **67**:2737–2741. https://doi.org/10.1109/TCSII.2020.2968376.
11. Yuan Q, Huang B, Zhang J. *et al.* Detecting phishing scams on Ethereum based on transaction records. In: *2020 IEEE International Symposium on Circuits and Systems (ISCAS), Seville, Spain, 12-14 October*, pp. 1–5. IEEE, 2020.
12. Chen L, Peng J, Liu Y. *et al.* Phishing scams detection in Ethereum transaction network. *ACM Trans Internet Technol* 2020; **21**:1–16.
13. Chen W, Guo X, Chen Z. *et al.* Phishing scam detection on Ethereum: Towards financial security for Blockchain ecosystem. In: *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI-20, Yokohama, Japan, January 7–15, 2021*, pp. 4456–4462. International Joint Conferences on Artificial Intelligence Organization, 2020.
14. Wen H, Fang J, Wu J. *et al.* Transaction-based hidden strategies against general phishing detection framework on Ethereum. In: *2021 IEEE International Symposium on Circuits and Systems (ISCAS), Daegu, Korea, 22-28 May*, pp. 1–5. USA: IEEE, 2021.
15. Ogundokun R, Arowolo M, Damaševičius R. *et al.* Phishing detection in blockchain transaction networks using ensemble learning. *Telecom*. 2023; **4**:279–297. https://doi.org/10.3390/telecom4020017.
16. Zhang D, Jinyin C. Blockchain phishing scam detection via multi-channel graph classification. In: *Blockchain and Trustworthy Systems: Third International Conference, BlockSys 2021, Guangzhou, China, 5–6 August*, pp. 241–256. Singapore: Springer, 2021.
17. Kim J, Sejong L, Kim Y. *et al.* Graph learning-based blockchain phishing account detection with a heterogeneous transaction graph. *Sensors*. 2023; **23**:463. https://doi.org/10.3390/s23010463.
18. Zhou X, Yang W, Tian X. Detecting phishing accounts on Ethereum based on transaction records and EGAT. *Electronics*. 2023; **12**:993. https://doi.org/10.3390/electronics12040993.
19. Etherscan. Available at https://etherscan.io/.
20. Hagberg A, Swart P, SChult D. *Exploring network structure, dynamics, and function using Networkx*, Technical report. Los Alamos

Downloaded from https://academic.oup.com/comjnl/article/67/12/3160/7746724 by National Institute of Technology Kurukshetra user on 05 February 2026

National Laboratory (LANL), Los Alamos, NM (United States), 2008.
21. Chawla NV, Bowyer KW, Hall LO. *et al.* SMOTE: Synthetic Minority Over-Sampling Technique. *J Artif Int Res* 2002; **16**:321–357.
22. Bastian M, Heymann S, Jacomy M. Gephi: an open source software for exploring and manipulating networks. In: *Proceedings of the International AAAI Conference on Weblogs and Social Media, San Jose, USA, 17-20 May*, pp. 361–362. California, USA: AAAI Press, 2009.
23. *The Igraph Software Package for Complex Network Research*, Available at https://igraph.org.
24. *Ethereum Lists*, Available at https://github.com/MyEtherWallet/ethereum-lists.
25. *Xblock*, Available at https://xblock.pro/#/.
26. *Ethereum Parity*, Available at https://www.parity.io/ethereum.
27. *Cryptoscamdb*, Available at https://cryptoscamdb.org.
28. Xiong A, Tong Y, Jiang C. *et al.* Ethereum phishing detection based on graph neural networks. *IET Blockchain*. 2023; **5**:1–9. https://doi.org/10.1049/blc2.12031.
29. Perozzi B, Al-Rfou R, Skiena S. DeepWalk: online learning of social representations. In: *The 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, New York, USA, 24–27 August*, pp. 701–710. New York, USA: ACM, 2014.
30. Preethi Krishna P, Sharada A. Word Embeddings - skip gram model. In: Gunjan, V. K., Garcia Diaz V, Cardona M, Solanki VK, Sunitha KVN (eds.), *ICICCT 2019 – System Reliability, Quality Control, Safety, Maintenance and Management, Hyderabad, 9–11 January*, pp. 133–139. Singapore: Springer, 2020. https://doi.org/10.1007/978-981-13-8461-5_15.
31. Grover A, Leskovec J. Node2vec: Scalable feature learning for networks. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16), San Francisco, California, USA, 13 - 17 August*, pp. 855–864, New York, United States: Association for Computing Machinery, 2016.
32. Rozemberczki B, Englert P, Kapoor A. *et al.* Pathfinder discovery networks. In: *WWW '21: The Web Conference 2021, Ljubljana, Slovenia, 19 - 23 April*, pp. 254–2558. New York, United States: Association for Computing Machinery, 2021.
33. He K, Zhang X, Ren S. *et al.* Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV), Santiago, Chile, 7-12 December*, pp. 1026–1034. USA: IEEE, 2015.
34. Cai T, Luo S, Xu K. *et al.* Graph-norm: a principled approach to accelerating graph neural network training. In: *Proceedings of the 38th International Conference on Machine Learning, PMLR 139, virtual, 18-24 July*, pp. 1204–1215. Harvard: MLR Press, 2020.
35. *AnomalyDetectionNtuplizer*, Available at https://github.com/aloeliger/anomalyDetectionNtuplizer.
36. Zhang X, He Y, Brugnone N. *et al.* MagNet: a neural network for directed graphs. In: *Advances in Neural Information Processing Systems, Virtual, 6–14 December*, pp. 27003–27015. Curran Associates, Inc., 2021.
37. Fey M, Lenssen JE. Fast graph representation learning with PyTorch geometric. 2019; Available at https://arxiv.org/abs/1903.02428.
38. van der Maaten L, Hinton G. Visualizing data using t-SNE. *J Mach Learn Res*. 2008; **9**:2579–2605.

Downloaded from https://academic.oup.com/comjnl/article/67/12/3160/7746724 by National Institute of Technology Kurukshetra user on 05 February 2026